

# 网络安全

---

Violet

2026-04-10



# Chapter 0

## 目 录

| I         | 密码学基础                           |
|-----------|---------------------------------|
| <b>1.</b> | <b>密码学概论 ····· 7</b>            |
| 1.1.      | 什么是密码学 ····· 7                  |
| 1.2.      | 核心安全目标 ····· 7                  |
| 1.3.      | 核心概念 ····· 7                    |
| 1.4.      | 攻击者模型与安全边界 ····· 8              |
| 1.5.      | 密码学发展简史 ····· 9                 |
| 1.6.      | 工程基础知识: XOR、字节序与编码 ····· 9      |
| 1.6.1.    | 异或 (XOR) ····· 9                |
| 1.6.2.    | 字节序 (Endianness) ····· 9        |
| 1.6.3.    | 编码 (Encoding) 与密文表示 ····· 10    |
| 1.7.      | 常用工具与实现建议 ····· 10              |
| 1.7.1.    | 命令行与库 ····· 10                  |
| 1.7.2.    | 示例: 推荐使用 AEAD (Python) ····· 10 |
| 1.8.      | 随机数安全 ····· 11                  |
| 1.8.1.    | 随机数分类 ····· 11                  |
| 1.8.2.    | 常见高风险误用 ····· 11                |
| 1.8.3.    | 工程建议 ····· 11                   |
| 1.9.      | 算法与场景速查 (统一选型) ····· 11         |
| 1.10.     | 常见易错点清单 ····· 12                |
| <b>2.</b> | <b>哈希函数与消息认证 ····· 13</b>       |
| 2.1.      | 哈希函数概述 ····· 13                 |
| 2.1.1.    | 核心特性 ····· 13                   |
| 2.1.2.    | 辨析: 单向哈希 vs 双向加密 ····· 13       |
| 2.2.      | 主流哈希算法 ····· 14                 |
| 2.2.1.    | 简要发展历程 ····· 14                 |
| 2.2.2.    | MD5 ····· 14                    |
| 2.2.3.    | SHA-1 ····· 14                  |
| 2.2.4.    | SHA-2 ····· 14                  |
| 2.2.5.    | SHA-3 (Keccak) ····· 14         |
| 2.2.6.    | SM3 ····· 14                    |
| 2.3.      | 哈希算法的安全性分析 ····· 15             |

- 2.3.1. 三类基础安全性 ····· 15
- 2.3.2. 碰撞攻击 ····· 16
- 2.3.3. 长度扩展与结构性风险 ····· 16
- 2.4. 哈希算法的应用 ····· 16**
- 2.4.1. 推荐应用场景 ····· 16
- 2.4.2. 禁止或不推荐场景 ····· 16
- 2.4.3. 彩虹表、弃用清单与迁移策略 ····· 16
- 2.4.4. 加盐哈希与 KDF ····· 17
- 2.4.5. 其他重要实践 ····· 17
- 3. 对称加密 ····· 18**
- 3.1. 对称算法概述 ····· 18**
- 3.1.1. 核心特性 ····· 18
- 3.1.2. 和非对称加密的对比 ····· 18
- 3.1.3. 简要发展史 ····· 18
- 3.2. 流密码 ····· 19**
- 3.2.1. 工作原理与关键要求 ····· 19
- 3.2.2. 代表算法 ····· 19
- 3.3. 分组密码 ····· 19**
- 3.3.1. 分组密码与流密码对比 ····· 19
- 3.3.2. DES 与 3DES ····· 20
- 3.3.3. AES（重点） ····· 20
- 3.3.4. SM4 ····· 21
- 3.3.5. 对称加密工作模式 ····· 21
- 3.3.6. 填充方式与安全风险 ····· 22
- 3.3.7. 常见攻击原理与风险提示 ····· 22
- 3.3.8. 生产弃用算法与配置清单 ····· 22
- 3.3.9. 主流场景行业最佳实践 ····· 23
- 4. 非对称加密 ····· 24**
- 4.1. 非对称算法概览 ····· 24**
- 4.1.1. 核心特性 ····· 24
- 4.1.2. 加密 vs 签名：本质辨析 ····· 24
- 4.1.3. 适用与禁用场景 ····· 24
- 4.2. 主流算法概览 ····· 25**
- 4.2.1. 简要发展脉络 ····· 25
- 4.2.2. RSA ····· 25
- 4.2.3. ECC（椭圆曲线密码） ····· 25
- 4.2.4. SM2 ····· 26
- 4.2.5. 签名专用算法 ····· 26
- 4.3. 非对称算法的核心概念 ····· 26**
- 4.3.1. 填充方式详解 ····· 26

---

|                              |           |
|------------------------------|-----------|
| 4.3.2. 数字签名方案详解与选型 .....     | 26        |
| 4.3.3. 常见攻击原理、影响与防御 .....    | 27        |
| 4.3.4. 其他重要概念 .....          | 27        |
| <b>4.4. PKI 公钥基础设施 .....</b> | <b>27</b> |
| 4.4.1. PKI 关键角色 .....        | 27        |
| 4.4.2. 证书校验核心要点 .....        | 27        |
| 4.4.3. 行业最佳实践 .....          | 28        |



# 密码学基础

|    |                 |    |
|----|-----------------|----|
| 1. | 密码学概论 .....     | 7  |
| 2. | 哈希函数与消息认证 ..... | 13 |
| 3. | 对称加密 .....      | 18 |
| 4. | 非对称加密 .....     | 24 |

# Chapter 1

## 密码学概论

### 1 什么是密码学

密码学 (Cryptography) 是网络安全的核心支柱，旨在研究如何将信息进行“伪装”，以防止未授权的第三方获取或篡改。现代密码学不再仅仅是关于“秘密书写”的艺术，而是一门基于严谨数学（如数论、计算复杂度理论、概率论）的科学体系。

#### NOTE

密码学的核心本质是：在不安全的环境中建立信任。

### 2 核心安全目标

在信息安全领域，密码学通常被要求实现以下四个核心目标（即 CIA + A/N 模型）：

| 目标                      | 描述                 | 对应的威胁攻击 |
|-------------------------|--------------------|---------|
| 机密性 (Confidentiality)   | 确保信息不被未授权者获取       | 窃听、数据泄露 |
| 完整性 (Integrity)         | 确保信息在传输或存储中未被篡改    | 篡改、伪造数据 |
| 真实性/认证性 (Authenticity)  | 确认通信主体的身份或消息来源     | 身份冒充    |
| 不可否认性 (Non-repudiation) | 发送方或接收方无法否认其已执行的操作 | 事后抵赖    |

#### TIP

密码学并不能直接提供“可用性 (Availability)”，但它是构建高可用系统的安全基础。

### 3 核心概念

#### 基本术语与数学抽象

密码学的操作过程可以抽象为一套标准符号体系：

- 明文 (Plaintext,  $P$ )：原始、可读的信息
- 密文 (Ciphertext,  $C$ )：加密后、不可读的信息
- 密钥 (Key,  $K$ )：控制加解密过程的关键参数
- 加密算法 (Encryption,  $E$ )：将明文转换为密文的数学函数
- 解密算法 (Decryption,  $D$ )：将密文还原为明文的数学函数

其基本关系可表示为：

$$C = E_{k(P)}$$
$$P = D_{k(C)} = D_{k(E_{k(P)})}$$

柯克霍夫原则 (Kerckhoffs' s Principle)

这是现代密码学的一条金科玉律。由奥古斯特·柯克霍夫在 19 世纪提出：

“即使密码系统的所有细节（除了密钥）都已公开，系统也应该是安全的。”

INFO

这与“通过隐藏实现的安全性”（Security through obscurity）形成鲜明对比。现代安全社区坚信：只有经过公开审查和大规模攻防验证的算法才是真正可靠的。

密码学原语分类

为了实现上述安全目标，密码学发展出了三大核心组件（原语），也是后续章节学习的重点：

| 组件类型               | 核心作用             | 关键词         |
|--------------------|------------------|-------------|
| 哈希函数 (Hash)        | 提供信息的“指纹”，确保完整性  | 单向、固定长度、抗碰撞 |
| 对称加密 (Symmetric)   | 提供高效的机密性保护       | 同一密钥、分组/流密码 |
| 非对称加密 (Asymmetric) | 提供密钥分发、身份认证和数字签名 | 公私钥对、计算密集型  |

术语与格式统一约定

为减少跨章节理解成本，建议整篇笔记保持以下统一约定：

- 安全目标：机密性、完整性、认证性、不可否认性
- 符号：明文 `m`、密文 `c`、密钥 `k`、摘要 `h`、签名 `s`、随机数 `nonce`
- 接口表达：优先使用 `Enc/Dec`、`Sign/Verify`、`MAC/VerifyMAC`
- 工程术语：将“随机数”细分为 `nonce`、`IV`、`salt`，避免混用

TIP

写代码时建议把参数名也与文档一致（如 `nonce`、`aad`、`tag`），可显著降低实现误读概率。

4 攻击者模型与安全边界

唯密文攻击 (COA)

攻击者只能获得密文，尝试通过统计规律或结构特征推测明文与密钥。

已知明文攻击 (KPA)

攻击者掌握部分明文-密文对，尝试利用映射关系还原密钥或构造更多明文。

选择明文攻击 (CPA)

攻击者可提交任意明文并获取其密文结果，重点分析算法在可控输入下的可区分性。



### 选择密文攻击 (CCA)

攻击者可提交任意密文并获取解密反馈，若协议错误处理不当，风险会显著上升。

**WARNING**

现代密码协议一般要求至少满足 CPA 安全；用于开放网络环境时，通常要求具备 CCA 安全（例如 AEAD、RSA-OAEP）。

## 5 密码学发展简史

| 阶段   | 时间           | 技术特征          | 典型代表            |
|------|--------------|---------------|-----------------|
| 古典密码 | 古代~19 世纪末    | 手工代换与置换       | 凯撒、维吉尼亚         |
| 近代密码 | 20 世纪初~1950s | 机械/机电密码机      | Enigma、Vernam   |
| 现代密码 | 1949 年至今     | 数学化、可证明安全、标准化 | DES/AES/RSA/ECC |

里程碑事件通常包括：

- 1949：Shannon 建立信息论与保密通信理论基础
- 1976：Diffie-Hellman 提出公钥思想
- 1977：RSA 提出并推动工程落地
- 2001：AES 成为分组密码主流标准

## 6 工程基础知识：XOR、字节序与编码

### 6.1 异或 (XOR)

异或是密码算法最常见的底层操作之一，核心性质：

$$A \oplus B \oplus B = A$$

这也是流密码可“同算法加解密”的数学基础。

```
1 def xor_encrypt_decrypt(data: bytes, key: bytes) -> bytes:
2     return bytes([b ^ key[i % len(key)] for i, b in enumerate(data)])
3
4 key = b"secret"
5 plaintext = b"Hello World"
6 ciphertext = xor_encrypt_decrypt(plaintext, key)
7 decrypted = xor_encrypt_decrypt(ciphertext, key)
8 print(decrypted)
```

**DANGER**

XOR 本身不是安全算法；真正安全性来自“高质量密钥流 + 不复用 nonce/计数器 + 认证保护”。

### 6.2 字节序 (Endianness)

多字节整数在内存中的存放顺序会影响跨语言和跨平台加解密结果：

- 大端序：高位字节在低地址
- 小端序：低位字节在低地址

在协议实现、逆向分析和二进制签名校验中，字节序不一致是常见 bug 来源。

```
1 import struct
2
3 num = 0x12345678
4 be = struct.pack(">I", num) # b'\x12\x34\x56\x78'
5 le = struct.pack("<I", num) # b'\x78\x56\x34\x12'
6 print(be, le)
```

 Python

## 6.3 编码 (Encoding) 与密文表示

编码不是加密，常见用途是“传输可表示性”：

- Hex：调试友好，体积膨胀约 2 倍
- Base64：传输友好，体积膨胀约 33%
- UTF-8：文本编码标准，不提供保密性

```
1 import base64
2
3 data = b"secret"
4 hex_data = data.hex() # 736563726574
5 b64_data = base64.b64encode(data).decode() # c2VjcmlV0
6 print(hex_data, b64_data)
```

 Python

### TIP

任何“看起来像乱码”的内容都未必是密文，也可能只是编码结果。工程审计时要先区分“编码”与“加密”。

## 7 常用工具与实现建议

### 7.1 命令行与库

- OpenSSL：协议与证书链路排查首选工具
- Python `cryptography`：现代 Python 密码学主力库
- Java JCE/JCA：企业 Java 生态标准接口
- Go `crypto/*`：标准库覆盖完善，适合构建基础设施

### 7.2 示例：推荐使用 AEAD (Python)

```
1 from cryptography.hazmat.primitives.ciphers.aead import AESGCM
2 import os
3
4 key = AESGCM.generate_key(bit_length=128)
5 aesgcm = AESGCM(key)
6 nonce = os.urandom(12)
7 aad = b"header"
8 plaintext = b"hello world"
```

 Python

```
9
10 ciphertext = aesgcm.encrypt(nonce, plaintext, aad)
11 recovered = aesgcm.decrypt(nonce, ciphertext, aad)
12 print(recovered)
```

**WARNING**

教学和演示时可展示 ECB/CBC 的机制，但生产代码应优先给出 AEAD 示例，避免错误迁移到真实系统。

## 8 随机数安全

随机数质量直接决定密钥安全上限。弱随机数会让再强的算法也失效。

### 8.1 随机数分类

| 类型               | 特点           | 适用性           |
|------------------|--------------|---------------|
| TRNG（真随机）        | 基于物理噪声，熵来源硬件 | 高安全熵源         |
| PRNG（伪随机）        | 确定性生成，可复现    | 仿真/非安全场景      |
| CSPRNG（密码学安全伪随机） | 不可预测、可抗状态恢复  | 密钥/nonce/令牌生成 |

### 8.2 常见高风险误用

- 使用 `rand()` 或时间戳直接生成密钥
- 多进程/容器冷启动导致熵池不足
- 复用 nonce、IV 或计数器初值
- 在日志中输出随机种子或密钥材料

### 8.3 工程建议

1. 密钥、nonce、token 一律来自 CSPRNG
2. 建立唯一性约束，监控 nonce 重复率
3. 对关键密钥生成流程做审计与压测
4. 用 KMS/HSM 托管主密钥，避免应用层直接接触

**NOTE**

安全工程里常说：随机数是密码系统的地基。地基不稳，上层算法再先进也无法提供真实安全性。

## 9 算法与场景速查（统一选型）

| 目标   | 推荐方案                                | 避免方案           | 备注         |
|------|-------------------------------------|----------------|------------|
| 口令存储 | Argon2id / bcrypt / scrypt / PBKDF2 | 无盐 SHA-256/MD5 | 口令哈希应慢且可升级 |
| 数据加密 | AES-GCM / ChaCha20-Poly1305         | ECB、裸 CBC      | 优先 AEAD    |
| 消息认证 | HMAC-SHA-256 / HMAC-SM3             | Hash(key  msg) | 避免长度扩展类风险  |

|      |                                 |           |          |
|------|---------------------------------|-----------|----------|
| 数字签名 | Ed25519 / ECDSA / RSA-PSS / SM2 | 裸 RSA、弱参数 | 按合规和生态选择 |
| 密钥交换 | X25519 / ECDHE / SM2 密钥交换       | 静态弱参数 DH  | 优先前向保密   |

## 10 常见易错点清单

- 1. 把编码（Base64/Hex）误当加密
- 2. 把哈希误当“可逆加密”
- 3. 复用 nonce 或 IV
- 4. 使用未认证加密却忽略篡改风险
- 5. 私钥与业务代码同仓库存放
- 6. 证书校验只校验链，不校验用途与域名
- 7. 生产环境仍保留 MD5/SHA-1/RC4/3DES 兼容路径

WARNING

密码系统上线前，建议至少进行一次“参数与配置安全审计”而不仅是功能测试。

# Chapter 2

## 哈希函数与消息认证

### 1 哈希函数概述

哈希函数 (Hash Function) 是将任意长度输入映射为固定长度摘要的函数，常记为：

$$h = H(m)$$

其中， $m$  是消息， $h$  是摘要 (digest)。在密码学场景中，哈希函数通常强调 **不可逆** 与 **抗攻击**，不仅仅是“算得快”。

#### 1.1 核心特性

密码学哈希函数的核心安全目标包括：

- **确定性**：同一输入总是得到同一输出
- **高效性**：可在可接受时间内完成计算
- **抗原像**：给定  $h$ ，难以找到  $m$  使得  $H(m) = h$
- **抗第二原像**：给定  $m_1$ ，难以找到  $m_2 \neq m_1$  且  $H(m_2) = H(m_1)$
- **抗碰撞**：难以找到任意  $m_1 \neq m_2$  使得  $H(m_1) = H(m_2)$
- **雪崩效应**：输入微小变化会导致输出显著变化

#### NOTE

“抗碰撞”最强、成本也最高。工程中常用“位数足够长 + 选用未被攻破算法”来降低风险。

#### 1.2 辨析：单向哈希 vs 双向加密

| 维度   | 单向哈希                  | 双向加密                 |
|------|-----------------------|----------------------|
| 是否可逆 | 不可逆（理论上无法还原明文）        | 可逆（持有密钥可解密）          |
| 输入输出 | 任意长度 -> 固定长度摘要        | 明文 <-> 密文（长度通常相关）    |
| 核心目的 | 完整性校验、指纹、签名前处理        | 保密传输与存储              |
| 密钥需求 | 一般不需要密钥（HMAC 例外）      | 必须有密钥                |
| 典型算法 | SHA-256 / SHA-3 / SM3 | AES / ChaCha20 / SM4 |

#### WARNING

常见误区：把哈希当“加密”使用。哈希不能解密，也不能直接提供机密性。

## 2 主流哈希算法

### 2.1 简要发展历程

- 1990s: MD4/MD5 流行，后续逐步发现碰撞风险
- 1995: SHA-1 发布，曾长期广泛部署
- 2001: SHA-2 (SHA-224/256/384/512) 发布，成为主流
- 2012: Keccak 竞赛胜出，标准化为 SHA-3
- 2010s: SM3 在国内标准体系中广泛采用

### 2.2 MD5

- 摘要长度: 128 bit
- 设计背景: Rivest 于 1992 年提出，基于 Merkle-Damgard 结构
- 现状: 已被实用碰撞攻击击破，不可用于安全相关签名或证书

典型风险: 攻击者可构造两份不同文件得到相同 MD5，导致“指纹一致但内容不同”。

### 2.3 SHA-1

- 摘要长度: 160 bit
- 历史地位: 曾用于 TLS、代码签名、Git 对象标识等
- 现状: 已出现实际可行碰撞，安全场景应弃用

### 2.4 SHA-2

- 家族: SHA-224 / SHA-256 / SHA-384 / SHA-512
- 结构: 迭代压缩 (Merkle-Damgard 系)
- 现状: 截至目前仍是工程主力，SHA-256/512 使用广泛

#### TIP

一般业务优先选 SHA-256；对高性能 64 位平台可考虑 SHA-512/256 等变体。

### 2.5 SHA-3 (Keccak)

- 摘要长度: 常见 224/256/384/512 bit
- 核心结构: Sponge (海绵) 结构，区别于传统 Merkle-Damgard
- 优势: 设计路径独立于 SHA-2；天然支持 XOF (如 SHAKE)

#### NOTE

SHA-3 不是“替代 SHA-2 的紧急补丁”，更像是并行体系，为算法多样性与长期稳健性提供保障。

### 2.6 SM3

- 摘要长度: 256 bit
- 标准背景: 我国商用密码哈希标准，常见于国密体系
- 应用生态: 与 SM2 (签名/密钥交换) 和 SM4 (分组加密) 配套使用

## SM3 发展脉络（简）

- 2000s：国内开始推进自主密码算法体系
- 2010：发布 SM3 相关国家/行业标准
- 后续：在政务、金融、关键基础设施等场景持续落地

## SM3 详细算法解析

SM3 采用分组迭代方式，整体流程可拆为 5 步：

1. 消息填充：先补 1，再补若干 0，最后附加消息长度，使总长度满足分组要求
  2. 分组：按 512 bit 划分消息块
  3. 消息扩展：每块扩展为  $W[0..67]$  和  $W'[0..63]$
  4. 压缩迭代：维护 8 个 32 位工作寄存器并进行 64 轮更新
  5. 输出：与上一链接变量异或得到新的链接变量，最终输出 256 bit 摘要
- 常见符号关系（示意）如下：

$$W'[j] = W[j] \oplus W[j + 4]$$

$$V_{i+1} = CF(V_i, B_i)$$

其中， $B_i$  为第  $i$  个消息分组， $CF$  为压缩函数。

### Algorithm

SM3 压缩函数（理解版）：

- 输入：上一轮链接变量  $V_i$ 、当前消息块  $B_i$
- 执行：
  - 进行消息扩展，得到  $W$  与  $W'$
  - 重复 64 轮布尔运算、模加、循环左移
  - 更新 A..H 八个寄存器
- 输出： $V_{i+1} = V_i \oplus (A \parallel B \parallel C \parallel D \parallel E \parallel F \parallel G \parallel H)$

### TIP

学习“详细算法”时，建议按“填充 → 扩展 → 轮函数 → 链接变量更新”四层结构记忆，而不是死背常量表。

## 3 哈希算法的安全性分析

### 3.1 三类基础安全性

| 性质    | 攻击目标               | 理想攻击复杂度             |
|-------|--------------------|---------------------|
| 抗原像   | 给定 $h$ 找 $m$       | 约 $2^n$             |
| 抗第二原像 | 给定 $m_1$ 找 $m_2$   | 约 $2^n$             |
| 抗碰撞   | 找任意 $m_1 \neq m_2$ | 约 $2^{(n/2)}$ （生日界） |

若哈希输出长度为  $n$  位，则碰撞攻击理论下界约为：

$$O(2^{\frac{n}{2}})$$

这就是为何摘要长度不能太短。

## 3.2 碰撞攻击

常见碰撞攻击路径：

- **随机碰撞搜索**：依赖生日悖论，复杂度约  $2^{\frac{n}{2}}$
- **构造型碰撞**：利用算法结构缺陷，远低于理论复杂度
- **选择前缀碰撞**：攻击者可控制两段前缀并构造同摘要后缀

工程影响：

- 伪造“看似相同指纹”的文件对
- 干扰签名/证书信任链（若系统仍依赖弱哈希）
- 造成审计与取证混淆

## 3.3 长度扩展与结构性风险

对某些 Merkle-Damgard 结构算法，若协议设计不当，可能出现长度扩展攻击：

- 错误用法： `Hash(key || msg)` 直接当作认证码
- 正确做法：使用 HMAC（如 HMAC-SHA-256 / HMAC-SM3）

### WARNING

“哈希 + 拼接密钥”不是通用安全方案。消息认证请优先使用标准 MAC 构造。

# 4 哈希算法的应用

## 4.1 推荐应用场景

- **完整性校验**：下载文件校验、镜像校验、区块数据校验
- **数字签名前处理**：先哈希再签名，提升效率并统一输入长度
- **消息认证**：基于 HMAC 的 API 签名、防篡改
- **内容寻址**：对象存储去重、版本指纹（注意碰撞与迁移策略）

## 4.2 禁止或不推荐场景

### DANGER

- 使用 MD5/SHA-1 作为安全签名依据
- 明文密码直接做 `SHA-256(password)` 后存库
- 把哈希当加密来“保护敏感数据”
- 自研 MAC、自定义“盐+哈希+截断”协议替代标准方案

## 4.3 彩虹表、弃用清单与迁移策略

### INFO

彩虹表是“预计算哈希反查表”，可显著降低弱口令破解成本。没有盐或盐过短时风险更高。



常见弃用清单（安全场景）：

- MD5（全面弃用）
- SHA-1（全面弃用）
- 无盐快速哈希存密码（全面弃用）
- 短摘要截断（如随意截成 32 64 bit）用于抗碰撞场景

迁移原则：

1. 新增数据全部切换到强算法
2. 老数据采用“登录时重哈希”或批量迁移
3. 保留算法版本字段，支持平滑升级

## 4.4 加盐哈希与 KDF

口令存储的基本原则不是“快”，而是 **故意变慢 + 增加内存成本**。

推荐策略：

- 每个用户独立随机盐（Salt）
- 可选服务端 Pepper（存放在 HSM/KMS 或独立配置）
- 使用专用口令 KDF，而非通用快速哈希

| 方案       | 特点         | 适用性               |
|----------|------------|-------------------|
| PBKDF2   | 迭代多次，成熟稳定  | 兼容性好，但抗 GPU 相对一般  |
| bcrypt   | 内置盐，成本参数可调 | 广泛可用，输出长度有限       |
| scrypt   | 强调内存硬性     | 抗并行硬件较好           |
| Argon2id | 当前口令哈希首选之一 | 综合抗侧信道与抗 GPU 能力较好 |

典型口令存储形式：

`$算法$参数$盐$派生值`

例如可记录：算法版本、时间成本、内存成本、并行度，便于后续升级。

## 4.5 其他重要实践

- **域分离**：不同业务用途使用不同前缀/标签，避免跨协议复用风险
- **规范化**：签名前先统一编码与字段顺序，避免“同义不同串”
- **常量时间比较**：比较摘要/标签时避免时间侧信道
- **密钥生命周期管理**：MAC 密钥需轮换、分级、可审计

### TIP

工程上最稳妥的策略是：

- 完整性：SHA-256 / SHA-3 / SM3
- 认证：HMAC-SHA-256 / HMAC-SM3
- 口令：Argon2id（优先）或 bcrypt / scrypt / PBKDF2

# Chapter 3

## 对称加密

### 1 对称算法概述

对称加密（Symmetric Encryption）指加密与解密使用同一把密钥（或可等价推导的密钥）。

$$c = E_{k(m)}, \quad m = D_{k(c)}$$

其中， $m$  为明文， $c$  为密文， $k$  为密钥。对称加密的核心目标是提供**机密性**，在现代系统中通常与完整性保护联合使用。

#### 1.1 核心特性

- **速度快**：适合大数据量、高吞吐加密
- **实现成熟**：硬件加速与软件库生态完善
- **密钥管理挑战大**：分发、轮换、隔离是主要难点
- **默认不提供完整性**：仅“加密”不等于“防篡改”

##### WARNING

只做“加密不认证”会产生可篡改风险。生产系统应优先选用 AEAD 模式（如 AES-GCM、ChaCha20-Poly1305）。

#### 1.2 和非对称加密的对比

| 维度   | 对称加密           | 非对称加密            |
|------|----------------|------------------|
| 密钥   | 同一密钥加解密        | 公钥加密/私钥解密或反向签名验证 |
| 性能   | 高（可用于大文件/流量）   | 低（常用于小数据或密钥交换）   |
| 典型用途 | 数据加密、磁盘加密、会话加密 | 密钥交换、数字签名、证书体系   |
| 工程组合 | 常作为数据面主力       | 常作为控制面与信任根       |

##### TIP

现代协议通常采用“混合加密”：先用非对称算法协商会话密钥，再用对称算法加密业务数据。

#### 1.3 简要发展史

- 1970s：DES 标准化，推动商用密码普及
- 1990s：3DES 作为过渡方案，弥补 DES 密钥长度不足
- 2001：AES 成为新一代主流分组密码标准
- 2000s 2010s：CTR/GCM 等高性能模式普及，AEAD 成为趋势

- 2010s 至今：ChaCha20-Poly1305 在移动端与无硬件加速环境广泛落地

## 2 流密码

流密码通过密钥流与明文逐字节/逐位组合得到密文，常见形式为异或：

$$c_i = m_i \oplus s_i$$

其中  $s_i$  是密钥流序列。

### 2.1 工作原理与关键要求

- 伪随机生成器根据密钥与随机数（nonce）产生密钥流
- 同一 (key, nonce) 绝不能复用
- 解密本质与加密相同（再异或一次）

#### DANGER

复用相同密钥流会导致“两次一密”（two-time pad）问题：攻击者可通过  $c1 \text{ xor } c2 = m1 \text{ xor } m2$  推断明文关系。

### 2.2 代表算法

#### RC4（历史算法，已弃用）

- 设计简单、曾部署广泛（WEP/TLS 早期版本）
- 存在统计偏差与多类已知攻击
- 生产禁用：现代系统不应继续使用 RC4

#### ChaCha20（现代推荐）

- 20 轮 ARX（加法-循环移位-异或）结构
- 软件实现高性能，尤其适合无 AES 指令集设备
- 常与 Poly1305 组合成 AEAD：ChaCha20-Poly1305

#### NOTE

在移动端和 IoT 场景，ChaCha20-Poly1305 往往比“无硬件加速 AES-GCM”更稳定高效。

## 3 分组密码

分组密码固定处理块（如 128 bit），基本记号：

$$C_j = E_{k(P_j)}$$

单个分组变换并不足以安全处理长消息，必须配合“工作模式（Mode of Operation）”。

### 3.1 分组密码与流密码对比

| 维度     | 分组密码                   | 流密码            |
|--------|------------------------|----------------|
| 处理单位   | 固定分组（如 128 bit）        | 按字节/按位连续处理     |
| 典型算法   | AES / SM4 / DES / 3DES | ChaCha20 / RC4 |
| 是否需要模式 | 需要（ECB/CBC/CTR/GCM 等）  | 通常自带流式机制       |
| 并行能力   | 取决于模式（CTR/GCM 强）       | 通常较强           |
| 填充需求   | 部分模式需要填充               | 通常不需要块填充       |

## 3.2 DES 与 3DES

### DES

- 分组长度：64 bit
- 密钥长度：有效约 56 bit
- 现状：密钥空间过小，已可被穷举攻击，生产禁用

### 3DES

- 思路：EDE（三次 DES）提高安全性
- 优点：兼容历史系统
- 问题：性能差、分组长度仍 64 bit、存在 Sweet32 风险背景
- 现状：逐步退场，仅限遗留兼容

## 3.3 AES（重点）

AES（Advanced Encryption Standard）是当前最主流分组密码之一。

### 基本参数

- 分组长度：128 bit
- 密钥长度：128 / 192 / 256 bit
- 轮数：10 / 12 / 14（随密钥长度增加）

### 结构与轮函数（AES-128）

AES 采用 SPN（Substitution-Permutation Network）结构。典型轮操作：

1. SubBytes（字节代换）
2. ShiftRows（行移位）
3. MixColumns（列混淆，最后一轮不执行）
4. AddRoundKey（轮密钥加）

#### Algorithm

AES-128（理解版）：

- 输入：16 字节明文块、128 bit 主密钥
- 先执行 AddRoundKey（初始轮）
- 重复 9 轮：SubBytes -> ShiftRows -> MixColumns -> AddRoundKey

- 最后一轮: SubBytes -> ShiftRows -> AddRoundKey
- 输出: 16 字节密文块

## | 工程实践重点

- 业务中很少“裸用 AES”，而是使用 AES-GCM、AES-CTR+MAC 等构造
- 优先使用经过验证的密码库与系统 API
- 在支持 AES-NI/ARMv8 Crypto Extension 的平台上性能优势明显

## 3.4 SM4

- 分组长度: 128 bit
- 密钥长度: 128 bit
- 应用定位: 国密体系中广泛使用，常见于政企与合规场景
- 工程建议: 优先使用 SM4-GCM / SM4-CCM 等认证加密构造

## 3.5 对称加密工作模式

### | 不安全或易误用模式（需重点避坑）

1. ECB: 相同明文块 -> 相同密文块，泄露模式结构，**禁止用于通用数据加密**
2. CBC: 只提供机密性，不提供完整性；若无独立 MAC，易遭篡改与填充预言机风险
3. CFB/OFB: 可流式处理，但仍需额外完整性保护；实现与重放控制要谨慎
4. CTR: 高性能可并行，但必须确保 nonce/counter 不复用，且需搭配 MAC 或 AEAD

#### WARNING

“CBC/CFB/OFB/CTR 本身都不是认证加密”。如果没有完整性保护，攻击者可能在不解密的情况下篡改数据。

### | 安全认证模式（AEAD，重点）

AEAD（Authenticated Encryption with Associated Data）同时提供机密性与完整性认证，可对附加数据 AAD（如协议头）做认证而不加密。

典型接口语义：

$$(c, tag) = Enc_{k(nonce, m, aad)}$$

$$m = Dec_{k(nonce, c, aad, tag)}$$

若认证失败，必须整体拒绝，不得返回“部分明文”。

### GCM (Galois/Counter Mode)

- 底层: AES-CTR + GHASH 认证
- 优点: 可并行、吞吐高、生态最广
- 关键要求: `(key, nonce)` 唯一；nonce 复用会造成严重灾难

### CCM (Counter with CBC-MAC)

- 底层: CTR 加密 + CBC-MAC 认证
- 特点: 实现路径不同于 GCM，常见于受限设备与部分无线协议

· 注意：参数配置（nonce/tag 长度）需严格按标准与协议约束

### ChaCha20-Poly1305

- 底层：ChaCha20 负责加密，Poly1305 负责认证
- 优点：软件性能稳定，抗时序实现更友好
- 场景：移动端、边缘设备、TLS/QUIC 常见首选之一

**TIP**

选型简化建议：

- 有硬件 AES 加速：优先 AES-GCM
- 无硬件 AES 加速或移动端优先：ChaCha20-Poly1305
- 国密合规：优先 SM4-GCM/SM4-CCM（按监管与标准实现）

## 3.6 填充方式与安全风险

分组模式中常见填充：PKCS7、ISO/IEC 7816-4、零填充（易误用）。

主要风险：

- 填充预言机（Padding Oracle）：错误信息或时序差异泄露填充正确性
- 数据截断与拼接：协议不完整认证导致可控篡改
- 不一致实现：跨语言/跨库填充规则不同造成互操作失败

防护要点：

1. 优先 AEAD，减少“填充 + 独立认证”的复杂性
2. 认证失败统一报错，不暴露细粒度差异
3. 严格区分“解密失败”和“业务校验失败”的外部可见行为

## 3.7 常见攻击原理与风险提示

| 风险类型     | 典型原因               | 后果        | 缓解建议                       |
|----------|--------------------|-----------|----------------------------|
| Nonce 复用 | 随机数生成错误/计数器回绕      | 明文泄露、认证失效 | 保证 nonce 唯一，加入监控           |
| 未认证加密    | 仅使用 CBC/CTR 不做 MAC | 密文可篡改     | 改为 AEAD 或 Encrypt-then-MAC |
| 弱算法遗留    | 历史兼容路径未清理          | 被动降级或主动利用 | 建立弃用基线与扫描                  |
| 密钥管理缺陷   | 硬编码、共享密钥、无轮换       | 大范围失陷     | KMS/HSM + 分级轮换             |

## 3.8 生产弃用算法与配置清单

- RC4（弃用）
- DES（弃用）
- 3DES（仅遗留兼容，不建议新系统）
- ECB（通用场景弃用）
- 固定 IV / 可预测 nonce（弃用）
- 自研加密协议与“魔改模式”（弃用）

**DANGER**

一条底线：不要发明自己的密码算法或协议。优先遵循公开标准与成熟实现。

## 3.9 主流场景行业最佳实践

### 通用文件加密

1. 使用 AEAD (AES-GCM 或 ChaCha20-Poly1305)
2. 每个文件使用独立随机数据密钥 (DEK)
3. 用主密钥 (KEK) 包裹 DEK, 主密钥存放 KMS/HSM
4. 文件头记录算法版本、nonce、tag 与密钥标识

### 移动端加密

1. 优先系统密钥容器 (Android Keystore / iOS Keychain)
2. 避免将长期密钥硬编码在应用包中
3. 无 AES 硬件加速设备优先 ChaCha20-Poly1305
4. 结合设备绑定、远程吊销与密钥轮换

### 数据库加密

1. 分层设计: 磁盘层 (TDE) + 字段层 (应用侧 AEAD)
2. 对高敏字段 (身份证号、银行卡号) 做字段级最小暴露
3. 保留可检索需求时使用“确定性加密/标记化”并进行风险评估
4. 严格审计密钥访问路径与解密操作

### 传输链路加密

1. 使用 TLS 1.2+ (建议 TLS 1.3)
2. 优先 AEAD 套件 (AES-GCM / ChaCha20-Poly1305)
3. 关闭弱套件与降级路径, 开启前向保密 (PFS)

#### NOTE

实战里“算法强度”只是基础, 真正决定安全上限的是密钥管理、协议配置与运维流程。

# Chapter 4

## 非对称加密

### 1 非对称算法概览

非对称加密 (Asymmetric Cryptography) 使用一对密钥：公钥 (public key) 和私钥 (private key)。

$$c = E_{pk}(m), \quad m = D_{sk}(c)$$

其中 `pk` 可公开分发, `sk` 必须严格保密。

#### 1.1 核心特性

- 密钥分离：公钥可公开，私钥仅持有者掌控
- 便于分发：相比对称密钥，跨组织协作更容易
- 计算开销较大：不适合直接加密大文件
- 可支持签名：天然适用于身份认证与不可否认性

#### WARNING

非对称算法通常用于“密钥交换、小数据加密、签名验证”。大数据加密应交给对称算法完成。

#### 1.2 加密 vs 签名：本质辨析

非对称体系中常见两类操作：

1. 公钥加密、私钥解密：解决机密性
2. 私钥签名、公钥验证：解决身份与完整性

签名关系可记为：

$$s = \text{Sign}_{sk}(m), \quad \text{Verify}_{pk}(m, s) = \text{true}$$

| 维度   | 公钥加密           | 数字签名               |
|------|----------------|--------------------|
| 目的   | 机密性            | 完整性 + 身份认证 + 不可否认性 |
| 密钥方向 | pk 加密 / sk 解密  | sk 签名 / pk 验签      |
| 谁可执行 | 任何拿到 pk 的人都可加密 | 仅私钥持有者可签名          |
| 典型场景 | 数字信封、密钥封装      | 证书签发、代码签名、交易签名     |

#### 1.3 适用与禁用场景

适用场景：

- 会话密钥协商 (TLS/SSH)



- 数字签名与证书体系
- 小规模敏感信息封装（如数据密钥 DEK）

禁用或不推荐：

**DANGER**

- 不要直接用 RSA/ECC 加密大文件
- 不要使用“裸 RSA”（无 OAEP/PSS 填充）
- 不要把私钥存放在源码、镜像或明文配置中
- 不要自定义签名拼接格式替代标准方案

## 2 主流算法概览

### 2.1 简要发展脉络

- 1977：RSA 提出，推动公钥密码进入工程实践
- 1990s：DSA/ECDSA 标准化，签名体系成熟
- 2000s 至今：ECC 因高安全强度/低密钥长度广泛应用
- 近年：EdDSA 在工程中快速普及，强调安全默认与实现稳健

### 2.2 RSA

- 数学基础：大整数分解困难问题
- 典型能力：加密、签名、密钥封装
- 工程建议：
  - 加密用 RSA-OAEP
  - 签名用 RSA-PSS
  - 新系统建议最少 2048 bit（长期可考虑 3072 bit）

### 2.3 ECC（椭圆曲线密码）

- 数学基础：椭圆曲线离散对数问题（ECDLP）
- 优势：较短密钥达到同等级安全强度，性能与带宽友好
- 典型能力：ECDH（密钥交换）、ECDSA（签名）、ECIES（混合加密体系）

#### 主流曲线与选型

| 曲线                | 类型               | 常见用途         | 选型建议         |
|-------------------|------------------|--------------|--------------|
| secp256r1 / P-256 | NIST Weierstrass | TLS、证书、ECDSA | 通用默认首选之一     |
| secp384r1 / P-384 | NIST Weierstrass | 高安全等级证书      | 对合规和强度有更高要求时 |
| Curve25519        | Montgomery       | X25519 密钥交换  | 现代协议优先       |
| Ed25519           | Edwards          | EdDSA 签名     | 签名首选之一       |
| secp256k1         | Koblitz          | 区块链生态        | 仅在特定生态按规范使用  |

**TIP**

通用业务可优先考虑：密钥交换用 X25519，签名用 Ed25519；若受既有 PKI 与合规约束，可选 P-256 + ECDSA。

## 2.4 SM2

- 定位：国密公钥算法，覆盖签名、密钥交换、公钥加密
- 应用场景：政企、金融、关键基础设施等国密合规环境
- 工程要点：与 SM3/SM4 组合部署，参数与证书格式需严格遵循标准

## 2.5 签名专用算法

### DSA

- 历史意义大，但新系统使用率明显下降
- 参数与随机数生成实现复杂，误用风险较高

### ECDSA

- 应用非常广泛（TLS、证书、区块链部分场景）
- 对随机数 **k** 极其敏感，重用或偏差会泄露私钥

### EdDSA (Ed25519 / Ed448)

- 现代签名方案，设计强调“安全默认”
- 确定性签名降低随机数质量依赖
- 实现一致性好，工程坑位相对更少

## 3 非对称算法的核心概念

### 3.1 填充方式详解

填充不是“可选增强”，而是公钥算法安全性的组成部分。

#### RSA 加密填充

1. PKCS1 v1.5：历史兼容用途，存在经典选择密文攻击背景
2. OAEP：现代推荐，抗选择密文能力更好

#### RSA 签名填充

1. PKCS1 v1.5 签名：兼容性高，但新系统不优先
2. PSS：现代推荐签名填充，安全证明更完备

**WARNING**

新系统原则：RSA 加密选 OAEP，RSA 签名选 PSS。

### 3.2 数字签名方案详解与选型

| 方案      | 优点            | 注意点         | 建议             |
|---------|---------------|-------------|----------------|
| RSA-PSS | 生态成熟、兼容广      | 密钥与签名较大     | 传统 PKI 和兼容优先场景 |
| ECDSA   | 签名短、性能好       | 实现必须严控随机数 k | 已有 ECC 生态可继续   |
| Ed25519 | 实现稳健、性能优、默认安全 | 需确认生态支持     | 新系统签名优先之一      |
| SM2 签名  | 满足国密合规        | 需配套国密证书与栈   | 国内合规场景优先       |

常见签名流程：

1. 对消息做标准化编码（防止同义不同串）
2. 对编码结果做哈希
3. 用私钥签名
4. 验签时严格复现同一规范化规则

### 3.3 常见攻击原理、影响与防御

| 攻击/风险              | 原理                  | 影响       | 防御要点             |
|--------------------|---------------------|----------|------------------|
| 小指数或参数错误           | 不安全参数导致可利用结构        | 可解密或伪造   | 采用标准参数与库默认       |
| Bleichenbacher 类攻击 | 基于填充错误回显的自适应查询      | RSA 解密泄露 | OAEP + 统一错误处理    |
| 签名随机数泄露            | ECDSA/DSA 的 k 重用或偏差 | 私钥恢复     | 高质量 RNG 或用 EdDSA |
| 侧信道攻击              | 时序/功耗/缓存泄露          | 密钥泄露     | 常量时间实现 + 硬件保护    |
| 证书链配置错误            | 验证逻辑不完整             | 中间人攻击    | 完整校证书链、用途、吊销状态   |

### 3.4 其他重要概念

- 前向保密（PFS）：会话密钥泄露隔离能力，TLS 推荐 ECDHE
- 密钥用途隔离：加密密钥、签名密钥、认证密钥分离
- 密钥轮换：设置生命周期并保留版本号，支持平滑迁移
- 证书透明度（CT）：发现异常证书签发，提升生态可审计性

## 4 PKI 公钥基础设施

PKI（Public Key Infrastructure）用于建立“公钥属于谁”的信任链。

### 4.1 PKI 关键角色

- Root CA：根信任锚，离线/高防护管理
- Intermediate CA：中间 CA，承担日常签发
- RA：注册机构，负责身份核验
- 证书持有者：服务端、个人、设备、代码签名主体
- 验证方：浏览器、客户端、网关、操作系统

### 4.2 证书校验核心要点

1. 证书链完整且可追溯到受信任根
2. 域名/主体匹配 (SAN 优先)
3. 证书用途正确 (Key Usage / EKU)
4. 有效期合法, 且检查吊销状态 (CRL/OCSP)
5. 算法与参数未过时 (禁用 SHA-1、弱密钥)

## 4.3 行业最佳实践

### ■ 数字信封 (Hybrid Encryption)

1. 随机生成对称数据密钥 DEK
2. 用 DEK (AEAD) 加密业务数据
3. 用接收方公钥加密 DEK (KEM/公钥加密)
4. 接收方私钥解封 DEK 后解密数据

### ■ 数字签名与身份认证

1. 服务端签发挑战值, 客户端私钥签名应答
2. 服务端用公钥或证书验签并校证书状态
3. 对关键操作追加时间戳、nonce、防重放机制

### ■ HTTPS / TLS 实践

- 优先 TLS 1.3; TLS 1.2 需禁用弱套件
- 密钥交换优先 ECDHE (PFS)
- 证书签名优先 ECDSA 或 RSA-PSS
- 开启 HSTS、OCSP Stapling (按场景) 与自动续期

### ■ SSH 实践

- 主机密钥优先 Ed25519 或 ECDSA (按合规)
- 禁用过时算法 (如依赖 SHA-1 的旧签名路径)
- 管理好 known\_hosts, 防止中间人替换主机指纹
- 用户认证优先公钥认证, 禁用口令直登与弱 MFA 配置

### ■ 各语言标准实现包 (常用)

| 语言      | 标准/主流实现                                            | 建议                                |
|---------|----------------------------------------------------|-----------------------------------|
| Java    | JCE/JCA (javax.crypto、java.security)               | 优先标准 Provider; 按需使用 Bouncy Castle |
| Go      | crypto/rsa、crypto/ecdsa、crypto/ed25519、crypto/x509 | 优先标准库, 减少自定义 ASN.1 处理             |
| Python  | cryptography (hazmat/x509)                         | 避免使用过时或无人维护库                      |
| Node.js | crypto (OpenSSL 后端)                                | 固定算法参数并关注运行时 OpenSSL 版本           |
| .NET    | System.Security.Cryptography                       | 使用平台托管密钥存储与证书 API                 |

|      |                         |                 |
|------|-------------------------|-----------------|
| Rust | ring / rustls / openssl | 优先内存安全实现与审计活跃项目 |
|------|-------------------------|-----------------|

**NOTE**

工程上最常见问题不是“算法不会用”，而是“参数、填充、证书校验、密钥管理”任一环节出现配置错误。